

B1 Coding Practical

Abstract. You are the engineer responsible for the development of the control system on an uncrewed underwater vehicle (UUV), designed to explore caves. Your colleagues have already designed the UUV's hardware, and modeled its dynamics, and now you are tasked with developing the controller that will regulate the depth of the vehicle.

Part 1: Codebase. The codebase for the project is available here: <https://github.com/jumenberger/b1-coding-practical-mt24>. Begin by creating a GitHub account (if you don't already have one) and forking the above repo to your own account.

Relevant videos include 2.7 Introduction to GitHub.

Part 2: Development Environment. You are free to choose any Integrated Development Environment (IDE) to develop your controller. The codebase is currently implemented in Python, so it is recommended to use *VS Code*. Specific sub-tasks include (a) configuring your IDE with the relevant extensions (e.g. to work with Python), and (b) ensuring that you have the relevant packages installed (there is a `requirements.txt` file in the repo, and it is strongly encouraged that you use a virtual environment).

Relevant videos include 1.2 VS Code: download and extensions, B.2 Python's inbuilt tools: pip and venv.

Part 3: Mission data. The depth profile of the cave to be explored by the UUV, and the corresponding target depth reference is available in the file `mission.csv`. Modify the `Mission` class, adding a method that returns an instance of the `Mission` class by extracting data from the `mission.csv` file. Before you start modifying your codebase, ensure that you are working on a **new branch**. Remember to make sufficiently regular commits as you

progress.

You may wish to make use of the pandas package and ChatGPT / Copilot to help you.

Part 4: Controller Development. Your team has decided to first test the efficacy of a proportional derivative (PD) feedback controller. The preliminary design is:

$$u[t] = K_P \cdot e[t] + K_D \cdot (e[t] - e[t - 1])$$

where $u[t]$ denotes the control action at (discrete) time t , $K_P = 0.15$ and $K_D = 0.6$ denote the proportional and derivative gains, respectively, and $e[t] = r[t] - y[t]$ denotes the error between the reference $r[t]$ and output (depth) $y[t]$, at time t . You will notice that the `ClosedLoop` class accepts both a `Submarine` object and a 'controller' on initialization. You are free to implement the controller in any way that you choose. Your code defining the controller should appear in a separate *module*, named *control*. You will also notice that the `simulate` method in the `ClosedLoop` class is incomplete: you must complete this method by producing the appropriate control action, given the observation and reference signals. You should test your controller implementation (adjusting the gain, if necessary) by making use of the `simulate_with_random_disturbances` and `plot_completed_mission` methods, in a *Jupyter notebook*.

Relevant videos include 1.3 VS Code: Jupyter Notebooks, 3.8 Introduction to objects and classes, 3.9 Modules, packages, and libraries.

Part 5: Merging new features. Once you're satisfied with your controller design and implementation, you should merge your new feature branch into the main branch of your repo. (You are free to do this locally, or via a pull request on GitHub). Either way, you should ensure that your updated main branch is pushed to GitHub.

Relevant videos include 2.5 Branching and fast-forward merges, 2.8 Publishing and pushing, and 2.12 GitHub Issues and Pull Requests.

Report. Once you have successfully pushed your controller to GitHub, you should write a brief (1 page) report to update your colleagues on the state of the software development. Focus particularly on: (a) What specific problem(s) did you solve? (b) What changes did you make to the codebase? Which functions / classes were modified? (Remember: keep this high-level; your colleagues can check the precise changes to the code via the GitHub repo). (c) Justify your design choices: why did you solve the problem the way that you did? e.g. Why did you use functions vs classes vs something different? (d) Suggest any possible improvements or directions for future work.

Relevant videos include 7.1 Writing technical reports.

Submission instructions. Please observe the following additional instructions and comments.

- The report is limited to 1 page, at 11 point font, double-line spacing
- 1 additional page may be used for appendices, if necessary
- The report must be submitted as a pdf
- The report will be marked out of 5.